Florence: Infrastructure - Supabase

1. Overview and Purpose

Supabase serves as the foundational Backend-as-a-Service (BaaS) for the Florence platform. It provides the core PostgreSQL database, user authentication via GoTrue and file storage. A critical feature utilised in this project is PostgreSQL Row Level Security (RLS) which ensures strict data segregation between patients, clinicians and administrators at the database level.

2. Configuration and Secrets Management

Dashboard URL: `https://supabase.com/dashboard/project/opltjtmmiuwbaikvlive`

Environment variables are required by the Python microservices and the Flutter frontend to communicate securely with Supabase.

Environment Variable	Description	Where it is stored	Required By
`SUPABASE_URL`	The unique project API URL	Vercel Environment Variables, Flutter `.env`	Backend, Frontend
`SUPABASE_ANON_KEY`	Public key for client-side authentication	Vercel Environment Variables, Flutter `.env`	Backend, Frontend
`SUPABASE_SERVICE_KEY`	Bypasses RLS for admin background tasks	Vercel Environment Variables	Backend Microservices

3. Technical Implementation Details

Database Schema Overview

The relational schema is designed to support a multi-tenant healthcare environment. Key tables include:

- **User Profiles:** `patient_profiles`, `clinician_profiles` and `admin_profiles` link directly to the core `auth.users` table.
- **Organisational Structure:** The `organisations` table manages clinics and hospitals, linking clinicians to specific facilities.
- **Health Monitoring:** `patient_monitor_data` stores biometric readings (glucose, blood pressure, BMI, cholesterol and HbA1c) while `patient_thresholds` stores personalised target ranges.
- **Lifestyle and Logs:** `daily_patient_logs` tracks meals and glucose impacts, `patient_activity_logs` tracks exercise and `disease_logs` manages medical history.
- **Medications:** A robust medication tracking system using `patient_medications`, `medication_dictionary` and `dosage_frequencies` alongside `medication_intake_logs` for adherence tracking.
- **AI and Chat:** `patient_chat_history` stores conversational context and `patient_recommendations` stores AI-generated health insights.
- **Documents:** `clinical_documents` tracks uploaded lab reports and medical files.

Row Level Security (RLS) and Data Privacy

RLS is enabled on all tables to enforce strict access control without relying solely on backend middleware.

- **Patients:** Can only read, insert, update and delete their own health data, logs and chat history. This is enforced by joining the target table with `patient_profiles` and verifying the `auth.uid()`.
- **Clinicians:** Can only view and manage data for patients explicitly assigned to them. Policies join `patient_profiles` with `clinician_profiles` to verify that the `clinician_id` matches the authenticated user.
- **Administrators:** Global admins possess full access across almost all tables to manage the platform, users and organisations. However, to enforce strict patient privacy, administrators are explicitly denied RLS access to highly sensitive private data, specifically `patient_chat_history`, `clinical_documents` and `automated_actions`.
- **Role Verification:** A secure PostgreSQL function `get_user_role()` reads the `role` claim from the Supabase JWT `app_metadata` to determine user permissions dynamically.

Database Functions

Custom PostgreSQL functions handle complex transactions securely:

- `create_patient_with_profile_and_thresholds` : Automatically generates a patient profile and seeds default clinical thresholds (glucose, HbA1c, BMI and blood pressure) upon registration.
- `create_clinician_with_profile` : Provisions a new clinician and links them to an organisation.
- `handle_new_user_settings` : A trigger function designed to automatically create a `user_settings` row for new users. *(Note: The function exists in the database, but the corresponding trigger is not currently bound to the* `auth.users` *table).*
- `delete_clinician_and_clean_up` : Safely removes a clinician, unassigns their patients and anonymises their name in existing clinical notes before deleting the underlying auth user.
- `get_all_table_names` : A utility function that returns a list of all ordinary tables within the `public` schema.
- `get_user_role` : A utility function that extracts the user's role from the JWT `app_metadata` to support dynamic permission checks.

Storage Buckets

Supabase Storage is utilised for managing user-generated and clinical files.

- `Bucket` : Stores patient and clinician profile pictures. RLS policies enforce that users can only upload, view and update files within their own `Profile_Picture/{auth.uid()}/` directory.
- `meal_photos` : Intended to store images of food for AI analysis. *(Note: This bucket currently lacks explicit RLS policies.)*
- `clinical-documents` : Stores uploaded PDFs or images of lab reports for biometric parsing. RLS policies restrict access to folders matching the authenticated user's UUID.

---

4. Billing, Limits and Day 2 Operations

- **Current Tier:** Free
- **Database Pausing:** If on the Free Tier, Supabase will pause the database after 7 days of inactivity. A credit card must be added in the Billing settings or upgrade to the Pro Tier to prevent service interruption.
- **Backups:** The Pro Tier includes daily automated backups and Point-in-Time Recovery. If remaining on the Free Tier, manual SQL dumps must be taken regularly via the Supabase dashboard or CLI.
- **Auth Limits:** Monitor the Monthly Active Users limit. The Free Tier allows 50,000 MAU which is sufficient for initial deployment but must be monitored as the user base grows.